



Slides for P3774R0

Rename `std::nontype`, and make it broadly useful

Document number:

P3775R0

Date:

2025-08-12

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:

Jan Schultke <janschultke@gmail.com>

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/nontype-slides.cow



IMPORTANT: This document has custom controls:

-   : go to the next slide
-   : go to previous slide

Rename `std::nontype`, and make it broadly useful

P3775R0

Jan Schultke, Bronek Kozicki, Tomasz Kamiński

Introduction

- P2472R3 added `std::nontype` constructors for `std::function_ref`
- merged into P0792R14, plenary-approved Varna 2023 for C++26
- `std::nontype` possibly obsoleted by `std::constant_wrapper` (P2781R9)
 - In C++26
- "non-type template parameter" term was removed by P2841R1 (C++26)
- in Sofia, two plausible directions discussed
 - replace `std::nontype` with `std::constant_wrapper`
 - rename `std::nontype` to `std::constant_arg`
 - latter option had more consensus

std::function_ref refresher

`std::function_ref` is a "function wrapper" similar to `std::function`, `std::move_only_function`, `std::copyable_function`, but without ownership.

`std::function_ref<R(Args...) cv noex>` stores:

- *bound-entity* — *BoundEntityType* (basically a *cv void**)
- *thunk_ptr* — `R(*) (BoundEntityType, Args...) noex`



NOTE: For illustration purposes, we'll ignore *cv*, *noex*, `invoke`.

std::nontype in std::function_ref

```
template<class F>          function_ref(F*);
template<class F>          function_ref(F&&);
template<auto f>           function_ref(nontype_t<f>);
template<auto f, class U>  function_ref(nontype_t<f>, U&&);
template<auto f, class T>  function_ref(nontype_t<f>, T*);
```

```
template<auto f>
function_ref(nontype_t<f>) {
    this->bound-entity = {};
    this->thunk_ptr = [] (BoundEntityType, Args&&... args) -> R {
        return f(std::forward<Args&&>(args)...);
    };
}
```

Alternatives considered

- Low-consensus:
 - `std::stateless` constructor tag
 - remove `std::nontype`, revisit in C++29
 - wait for P1045R1 `constexpr` function parameters
 - expose the *BoundEntityType*
- Plausible:
 - replace `std::nontype` with `std::constant_wrapper`
 - rename `std::nontype` (most consensus in Sofia)

I — Nullary functions

```
using status_t = int;
constexpr status_t perform_task() {
    // TODO: implement, this is a placeholder
    return STATUS_OK;
}

// OK, extracts &perform_task,
// r() calls perform_task
std::function_ref<status_t()> r = std::cw<&perform_task>;

// OK, but m() does not call perform_task;
// we create a wrapper which returns pre-computed result
std::move_only_function<status_t()> m = std::cw<&perform_task>;
```

II — Proper constant wrappers

```
struct Zero {  
    static constexpr int value = 0;  
    constexpr operator int() const { return value; }  
    constexpr int operator()() const { return value; }  
};  
  
// OK, calls Zero::operator()  
std::function_ref<int()> r = std::cw<Zero{}>;  
  
// OK, but simply returns 0  
std::move_only_function<int()> m = std::cw<Zero{}>;
```

III — Improper constant wrappers

```
struct Zero {  
    static constexpr int value = 0;  
    constexpr operator int() const { return value; }  
    constexpr int operator()() const { return value; }  
};  
  
// OK, calls Zero::operator()  
std::function_ref<int()> r = std::cw<Zero{}>;  
  
// error  
std::move_only_function<int(int)> m = std::cw<Zero{}>;
```

IV — Surrogate calls

```
constexpr int sqr(int x) {  
    return x * x;  
}  
  
// OK, extracts &sqr  
std::function_ref<int(int)> r = std::cw<&sqr>;  
  
// OK, surrogate function call  
// because of conversion to int (*)(int)  
std::move_only_function<int(int)> m = std::cw<&sqr>;
```

V — Function objects

```
struct Sqr {  
    constexpr int operator()(int x) const {  
        return x * x;  
    }  
};  
inline constexpr Sqr sqr;  
  
// OK, extracts sqr  
std::function_ref<int(int)> r = std::cw<sqr>;  
  
// error  
std::move_only_function<int(int)> m = std::cw<sqr>;
```

Proposed solution — `std::fn`

- C++26: rename `std::nontype_fn`, `std::nontype_t_fn_t`
- C++26: move decl. from `<utility>` to `<functional>`
- C++29: add call operator, extra functionality

```
// vision for C++29
constexpr int sqr(int x) { return x * x; }

function_ref<int(int)> ref = fn<sqr>;           // OK, same as C++26
views::transform(range, fn<sqr>);             // good for inlining
move_only_function<int(int)> m = fn<sqr>;      // OK, optimal
boost::function_ref<int(int)> b = fn<sqr>;     // OK, optimal
```

Discussion

- Do we agree `std::constant_wrapper` should not be used here?
- Do we agree on the name?
 - `std::constant_arg` had "consensus"
 - `std::fn` / `std::fn_t` proposed
 - `std::const_fn` / `std::const_fn_t` possible alternative
 - `std::fw` / `std::function_wrapper` possible alternative
- Do we agree on moving to `<functional>`?